# SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING) Department of Computer Science and Engineering

## ASSESSMENT TOOL 1 – Constraint-Based Problem Solving

| | | | |
|---|---|---|---|
| **Course Code:** | CSA06 | **Course Title:** | Design and Analysis of Algorithms |
| **CO Assessed:** | CO5– Naturalization (BL5,BL6) | **Assessment:** | Assessment Tool 1 – Constraint Based Problem Solving |
| **Weightage:** | 30% of CIA | **Total Marks:** | 100 |
| **CO5 Statement:** | *Analyze computational problems using backtracking techniques and complexity classes such as P, NP, NP-Hard, and NP-Complete for combinatorial problem solving.* | | |
| **Student Name:** | **JAYAKRISHNA.V** | **Reg. No.:** | **192421259** |
| **Section / Batch:** | **I** | **Date:** | **26.08.26** |

**Instructions to Students**

• Answer the following question. Total Marks: 100.
• Apply backtracking techniques systematically for combinatorial problem solving.
• Clearly classify problems under P, NP, NP-Hard, and NP-Complete categories wherever required. • Provide proper justification, state-space representation, and complexity analysis for all solutions. • Neat presentation and logical explanation are mandatory.

| Question Type | Focus Area | Questions |
|---|---|---|
| Constraint Based Problem Solving | Analyze combinatorial problems using backtracking and complexity classes such as P, NP, NP-Hard, and NP Complete. | Q1 |

## SECTION A – CONSTRAINT-BASED PROBLEM SOLVING

**Code of Conduct**

*I(JAYAKRISHNA.V/192421259) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the Student: _____JAYAKRISHNA.V _____

## RUBRICS FOR CALCULATION

| Criteria | Wt. | Level 4 – Exemplary | Level 3 – Proficient | Level 2 – Developing | Level 1 – Beginning |
|---|---|---|---|---|---|
| Problem Understanding | 25% | Clearly defines the problem and identifies all relevant constraints accurately | Identifies most constraints with minor gaps | Partial understanding; misses key constraints | Fails to identify constraints correctly |
| Constraint Analysis | 25% | Analyzes constraints effectively and prioritizes them logically | Provides reasonable analysis with some prioritization | Limited analysis; weak prioritization | No meaningful analysis or prioritization |
| Solution Development | 25% | Develops optimal and feasible solutions satisfying all constraints | Develops workable solutions with minor limitations | Solutions partially satisfy constraints | Solutions do not meet constraints |
| Application of Concepts | 15% | Effectively applies appropriate concepts, methods, or tools | Applies concepts with minor errors | Limited application; requires guidance | Unable to apply relevant concepts |
| Justification | 10% | Provides strong justification for decisions with logical reasoning | Provides justification with some clarity | Weak or unclear justification | No justification provided |

## Marks Evaluation:

| Criteria | Wt. | Level 4 – Exemplary | Level 3 – Proficient | Level 2 – Developing | Level 1 – Beginning |
|---|---|---|---|---|---|
| Problem Understanding | 25% | | | | |
| Constraint Analysis | 25% | | | | |
| Solution Development | 25% | | | | |
| Application of Concepts | 15% | | | | |
| Justification | 10% | | | | |
| **Total Marks (100):** | | | | | |

| Faculty In-charge | Course Coordinator | Head of Department |
|---|---|---|
| Name: | Name: | Name: |
| Signature: _____ | Signature: _____ | Signature: _____ |
| Date: _____ | Date: _____ | Date: _____ |

**1. Problem Understanding**

The **Hamiltonian Path Problem** is a classical graph traversal problem. Given a graph containing vertices and edges, the objective is to determine whether there exists a path that visits every vertex **exactly once**.

A Hamiltonian path must satisfy the following conditions:

- Every vertex is included in the path.
- No vertex is visited more than once.
- Consecutive vertices in the path must be connected by an edge.
- The path does not need to return to the starting vertex. If it returns to the starting vertex, it becomes a Hamiltonian cycle.

**Mathematical Formulation**

For a graph $G = (V, E)$, a path

$$P = [v_0, v_1, v_2, \dots, v_{n-1}]$$

is Hamiltonian if:

$$| P | = | V |$$

and

$$(v_i, v_{i+1}) \in E \text{ for } 0 \leq i < n - 1$$

with

$$v_i \neq v_j \text{ for all } i \neq j$$

**Input and Output Specification**

**Input:** An adjacency matrix representing the graph.

python

```
graph = [
   [0, 1, 1, 0],
   [1, 0, 1, 1],
   [1, 1, 0, 1],
   [0, 1, 1, 0]
]
```

**Output:**
text
Hamiltonian Path Exists
Path: 0 1 2 3
The path $0 \rightarrow 1 \rightarrow 2 \rightarrow 3$ visits all four vertices exactly once, and every consecutive pair of vertices has an edge.

---

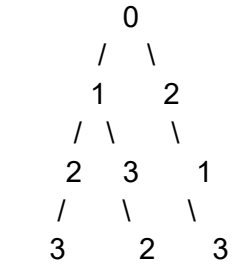
**2. Constraint Analysis**
Constraint checking is used to eliminate invalid paths while constructing the solution.

| Constraint Type | Mathematical Condition | Impact on Search Space |
|---|---|---|
| Vertex Uniqueness Constraint | $v_i \neq v_j$ for $i \neq j$ | Prevents the same vertex from being included more than once. |
| Edge Connectivity Constraint | graph$[u][v] = 1$ | Ensures that consecutive vertices in the path are directly connected. |
| Path Length Constraint | ( | P |
| Starting Vertex Constraint | $P = 0$ | Fixing the starting vertex reduces duplicate searches caused by different starting points. |
| Completion Constraint | All vertices are visited | Terminates the search successfully when a complete path is formed. |

The uniqueness and connectivity constraints significantly reduce the number of valid paths. Any candidate that contains a previously visited vertex or uses a nonexistent edge is rejected immediately.

---

**3. State-Space Representation and Pruning Techniques**
The search space can be represented as a **tree of possible paths**. Each level of the tree corresponds to selecting one vertex for the next position in the path.
For the graph shown in the program:
text
Level 0: Start with vertex 0




Each branch represents a possible choice for the next vertex.
**Branching Decisions**
At each recursion level:

- Select an unvisited vertex.
- Check whether it is connected to the previous vertex.

- Add it to the path if it satisfies the constraints.
- Recursively continue with the next position.
- Remove it if the selected branch does not lead to a solution.

**Pruning Rules**

The following pruning techniques are applied:

- **Visited Vertex Pruning:** Reject a vertex if it already appears in the current path.
- **Invalid Edge Pruning:** Reject a vertex if no edge exists between it and the previous vertex.
- **Early Success Exit:** Stop immediately when all vertices have been added to the path.
- **Backtracking:** Remove the most recently selected vertex when a branch becomes invalid.
- **Fixed Starting Vertex:** Begin from vertex 0 to reduce equivalent path searches.

These constraints prevent the algorithm from exploring paths that can no longer produce a valid Hamiltonian path.

---

**4. Implementation in Python**

python

```
graph = [
   [0, 1, 1, 0],
   [1, 0, 1, 1],
   [1, 1, 0, 1],
   [0, 1, 1, 0]
]

n = len(graph)
path = [-1] * n
path[0] = 0


def is_safe(vertex, position):
   # Check whether the vertex is connected to the previous vertex
   if graph[path[position - 1]][vertex] == 0:
      return False

   # Check whether the vertex has already been used
   if vertex in path[:position]:
      return False

   return True


def hamiltonian_path(position):
   # All vertices have been included
   if position == n:
```

```
        return True

    # Try every possible vertex
    for vertex in range(1, n):
        if is_safe(vertex, position):
            path[position] = vertex

            if hamiltonian_path(position + 1):
                return True

            # Backtrack and remove the vertex
            path[position] = -1

    return False


if __name__ == "__main__":
    if hamiltonian_path(1):
        print("Hamiltonian Path Exists")
        print("Path:", " ".join(map(str, path)))
    else:
        print("No Hamiltonian Path Exists")
```

**Important Functions**

**is_safe(vertex, position)**

This function verifies two conditions:

1. The candidate vertex is connected to the previously selected vertex.
2. The candidate vertex has not already been included in the path.

**hamiltonian_path(position)**

This recursive function:

1. Checks whether all vertices have been visited.
2. Iterates through possible candidate vertices.
3. Adds a safe vertex to the path.
4. Recursively searches for the next position.
5. Backtracks if the selected vertex does not lead to a solution.

---

**5. Execution and Testing**

**Sample Input Graph**

python

```
graph = [
    [0, 1, 1, 0],
    [1, 0, 1, 1],
    [1, 1, 0, 1],
    [0, 1, 1, 0]
]
```

The edges represented by this matrix include:
text
0 -- 1
0 -- 2
1 -- 2
1 -- 3
2 -- 3

**Execution Trace**

The algorithm starts from vertex 0.
text
hamiltonian_path(1)
Possible execution sequence:
text
1. Select vertex 1
   Path: 0 1

2. Select vertex 2
   Path: 0 1 2

3. Select vertex 3
   Path: 0 1 2 3

4. All vertices are included
   Hamiltonian path found

**Output**

text
Hamiltonian Path Exists
Path: 0 1 2 3

**Path Verification**

The generated path is:

$$0 \rightarrow 1 \rightarrow 2 \rightarrow 3$$

The path is valid because:

- Vertices 0, 1, 2, and 3 are all visited.
- No vertex is repeated.
- Edge $0 \rightarrow 1$ exists.
- Edge $1 \rightarrow 2$ exists.
- Edge $2 \rightarrow 3$ exists.

---

**6. Correctness Justification**

The algorithm is correct because it systematically explores every possible ordering of vertices beginning from the selected starting vertex.

At each stage, the algorithm only adds a vertex when:

- It is connected to the previous vertex.

- It has not already appeared in the path.

Therefore, every path maintained by the algorithm satisfies the required constraints.
If the recursion reaches position == n, every vertex has been selected exactly once, and the resulting path is a valid Hamiltonian path.
If a branch cannot be completed, the algorithm backtracks and tries another unused connected vertex. Since all valid alternatives are examined, the algorithm will find a Hamiltonian path whenever one exists.

---

**7. Complexity Analysis**

**Time Complexity**

In the worst case, the algorithm may examine every possible ordering of the vertices.
For $n$ vertices, the number of possible paths is approximately:

$$(n-1)!$$

because the starting vertex is fixed.
Therefore, the worst-case time complexity is:

$$O\big((n-1)!\big)$$

This is generally expressed as:

$$O(n!)$$

**Best-Case Complexity**

The best case occurs when a valid path is found immediately:

$$O(n)$$

This happens when the first sequence of valid choices forms a Hamiltonian path.

**Auxiliary Space Complexity**

The algorithm uses:

- The path array of size $n$.
- A recursion stack with maximum depth $n$.

Therefore, the auxiliary space complexity is:

$$O(n)$$

The adjacency matrix itself requires:

$$O(n^2)$$

memory.

---

**8. Complexity Class Categorization**

The Hamiltonian Path Problem belongs to the class **NP** because a proposed path can be verified in polynomial time.
To verify a candidate path:

1. Check that the path contains exactly $n$ vertices.
2. Check that no vertex is repeated.

3. Check that every consecutive pair has an edge.

These checks can be completed in polynomial time.

The decision version of the Hamiltonian Path Problem is **NP-Complete**. It is NP-Hard because several well-known NP-Complete problems, such as the Hamiltonian Cycle Problem, can be reduced to it in polynomial time.

Although the problem is computationally difficult in general, the backtracking algorithm provides a complete solution for small and medium-sized graphs.

---

## 9. Advantages and Limitations

### Advantages

- Simple and easy to understand.
- Guarantees a correct result.
- Uses constraint checking to avoid many invalid paths.
- Requires only linear auxiliary space apart from the graph representation.
- Can terminate early when a valid path is found.

### Limitations

- Worst-case running time is factorial.
- Performance decreases rapidly as the number of vertices increases.
- Backtracking may still explore a large number of candidate paths.
- It is not suitable for very large dense graphs without additional optimization.

## OUTPUT